Smart Ambulance Routing System

& Hospital Allocation System

Complete Slide-by-Slide Explanation + End-to-End Architecture Flow

BCS401 ADA | BCS402 OOCJ | BCS403 DBMS | BCS405B GT

Table of Contents

Shreya R Hipparagi · VTU 4th Semester CSE · 2025–26

github.com/ShreyaRHipparagi/Ambulance

Slide 1 — Title Slide

SLIDE 1

Project introduction, purpose, and academic coverage

What This Slide Shows

The title slide introduces the project to the audience — examiner, faculty, or peers. It immediately communicates what the project does (smart ambulance routing), who built it (4th Semester CSE student), and why it is academically relevant (4 subjects demonstrated end-to-end).

Design Decisions

- Dark navy (#0B1D3A) background creates a professional, premium look — avoids the 'default PPT template' look.
- Teal left stripe (#00B4D8) is a visual anchor that repeats on every dark slide — creates visual continuity.
- The ambulance emoji inside a circle immediately communicates the project domain without needing text.
- Four coloured subject badges (amber, green, purple, teal) are positioned prominently so an examiner sees all 4 subjects at a glance.
- The tagline is italicized to distinguish it visually from the main title without being too prominent.

What to Say in Viva (Slide 1)

Opening Statement

"This project is called Smart Ambulance Routing and Hospital Allocation System. It is a React-based web application that uses real Bangalore city map data — 13 GPS-verified locations and 23 road connections — to find the optimal hospital for an emergency patient. It simultaneously demonstrates all four of our 4th semester subjects: BCS401 Algorithm Design, BCS402 Advanced Java concepts, BCS403 DBMS, and BCS405B Graph Theory."

Academic Subjects Covered (Slide 1)

- BCS401 ADA — Motivates why an algorithm is needed (ambulance routing problem)
- BCS402 OOCJ — OOP structure of the React component architecture
- BCS403 DBMS — Database design problem (hospitals, patients, roads, allocations)
- BCS405B GT — City roads as a graph (nodes = intersections, edges = roads)

Slide 2 — What This Project Does

SLIDE 2

Problem statement vs solution — the core motivation

The Problem (Left Card)

- In real emergencies, dispatchers call the nearest hospital — but 'nearest' is based on intuition, not data.
- The nearest hospital may not handle the emergency type (e.g., no cardiac unit).
- It may be full — zero ICU beds or no general beds available.
- It may not be the fastest route if there is traffic congestion on that road.
- Manual decision-making under pressure leads to preventable delays — the golden hour is lost.

Our Solution (Right Card — 4 Steps)

1. **Match hospital:** Check if the hospital handles the specific emergency type (CARDIAC, TRAUMA, NEURO, BURNS, MATERNITY, GENERAL)
2. **Check resources:** Confirm ICU beds are available (if patient needs ICU) or general beds are available
3. **Find best route:** Run Dijkstra's algorithm on the city graph — weighted by road distance and live traffic multiplier
4. **Recommend:** Return the lowest-cost hospital as PRIMARY and second-lowest as BACKUP

Why These 4 Steps Matter

Without This System	With This System
Nearest hospital chosen blindly	Best-qualified hospital selected
No specialization check	Emergency type verified first
No bed availability check	Bed count checked in real-time
No traffic-aware routing	Dijkstra + live traffic routing
No backup hospital suggested	Primary + backup both provided

What to Say in Viva (Slide 2)

Key Point

"Our system does not just find the closest hospital — it finds the BEST hospital. It filters by emergency specialization, then checks bed availability, and only then runs Dijkstra's shortest-path algorithm on the city road graph. This ensures the recommended hospital can actually treat the patient AND is reachable in the minimum time."

Slide 3 — BCS401: Algorithm Design & Analysis

BCS401

Concepts, justifications, and complexity used in this project

Card 1: Problem Definition & Requirements Analysis

- Formally defined the problem: given a patient at location X with emergency type E, find hospital H* that minimises travel cost c(X, H*) subject to: H*.specializations contains E AND H*.bedsAvailable > 0.
- This is a constrained shortest-path problem on a weighted graph — it requires ADA-level thinking.
- Requirements Analysis: identified 3 constraints (type, beds, distance) and 2 outputs (primary + backup).

Card 2: Brute-Force vs Smart Search

Brute-Force O(H × V²)

- Check every possible hospital
- For each hospital: scan all paths
- No filtering — wastes computation
- Scales badly: 6 hospitals × 13 nodes

Smart: Filter + Dijkstra O((V+E)logV)

- Step 1: filter by specialization
- Step 2: filter by bed availability
- Step 3: Dijkstra only for candidates
- O(k(V+E)logV) where k "d 6 hospitals

Card 3: Why Dijkstra? (The Core Justification)

- Road edges have DIFFERENT distances (weights) — 1.2 km vs 7.2 km. BFS treats all edges as equal — WRONG for roads.
- Dijkstra finds the globally optimal shortest path in a weighted graph with non-negative weights — proven correct.
- A* improves on Dijkstra using a GPS heuristic: h(n) = Haversine distance to target. It prunes irrelevant nodes.
- BFS/DFS are used for educational visualization only — not for the actual recommendation.

Card 4: Time Complexity Comparison

```
BFS / DFS     :  O(V + E)          V=13, E=23  !'  O(36)   — unweighted traversal only
Dijkstra      :  O((V+E) log V)    V=13, E=23  !'  O(36 × 3.7) "H O(133)
A* Search     :  O(E) best case    with GPS heuristic — prunes ~40% of nodes
Binary Heap   :  insert O(log N), extractMin O(log N)  — backing Dijkstra & A*
```

What to Say in Viva (BCS401)

Examiner Question: Why not BFS?

"BFS finds the path with the fewest hops — not the shortest distance. In our city graph, a 2-hop path might be 12 km while a 3-hop path is only 3.5 km. BFS would pick the 2-hop path incorrectly. Dijkstra considers edge weights — real road distances — so it always finds the minimum-distance path. That is why we use Dijkstra."

Slide 4 — BCS402: Advanced Java / OOP Concepts

BCS402

Object-oriented design patterns used in the React + JS project

Row 1: Object-Oriented Programming (OOP)

- CityGraph class — encapsulates the adjacency list, addNode(), addEdge(), getNeighbors(), getEffectiveWeight().
- DijkstraEngine class — single responsibility: takes a graph and runs SSSP. No UI concerns.
- RecommendationService class — orchestrates the pipeline (filter !’ Dijkstra per hospital !’ sort !’ return).
- PriorityQueue class — pure min-heap with insert(), extractMin(), peek(), toArray(). Zero dependencies.
- Each class follows Single Responsibility Principle — one class, one job.

Row 2: Collections Framework (JavaScript equivalents)

- HashMap !’ JavaScript Map — O(1) lookup for nodes, adjacency lists, distance tables.
- ArrayList !’ JavaScript Array — neighbor lists, step capture arrays, history records.
- PriorityQueue !’ Custom binary min-heap — insert O(log N), extractMin O(log N).
- Set !’ JavaScript Set — visited node tracking in Dijkstra (prevents revisiting).

Row 3: Swing UI !’ React Component Model

- React components "a Swing JPanel — each component has state, renders UI, handles events.
- useState() "a class fields — stores form data, algorithm steps, route data.
- useEffect() "a componentDidMount/Update — fires on mount (map init, data fetch) and on state change.
- AppContext "a Singleton service — global state accessible from any component without prop drilling.
- Framer Motion "a Java animation timers — smooth page transitions and card entrance animations.

Row 4: JDBC !’ localStorage (Data Persistence)

- JDBC connects Java app to MySQL — localStorage connects React app to browser storage.
- Both provide: write (INSERT/UPDATE), read (SELECT), delete (DELETE) operations.
- localStorage['emergency_history'] "a audit log table — persists across sessions.
- localStorage['hospital_beds'] "a resource table — bed counts survive page refresh.

What to Say in Viva (BCS402)

Key Mapping

"Although this project is in JavaScript and React rather than Java Swing, every OOP concept from BCS402 is demonstrated: CityGraph is an encapsulated class with private state and public methods — exactly like a Java class. The RecommendationService uses dependency injection — it receives the graph and hospitals as constructor arguments, making it testable and reusable."

Slide 5 — BCS403: Database Management System

BCS403

Relational schema, ER diagram, 3NF normalization, and SQL queries

ER Diagram — 8 Entities, 3NF

- patients (patient_id PK, name, age, blood_group, contact)
- location_nodes (node_id PK, name, area, lat, lng)
- road_edges (edge_id PK, from_node FK, to_node FK, weight_km, traffic_multiplier)
- hospitals (hospital_id PK, name, address, contact, node_id FK)
- hospital_specializations (spec_id PK, hospital_id FK, emergency_type)
- hospital_resources (resource_id PK, hospital_id FK, icu_total, icu_avail, gen_total, gen_avail)
- emergency_cases (case_id PK, patient_id FK, source_location FK, emergency_type, needs_icu, timestamp)
- allocations (allocation_id PK, case_id FK, hospital_id FK, backup_hosp FK, route_cost_km, path_nodes)

3NF Normalization Proof

```
1NF: All attributes atomic — no multi-valued fields (specializations split to own table ' )
2NF: No partial dependencies — all non-key attributes depend on full primary key '
3NF: No transitive dependencies:
     patients: patient_id !’ name, age (no non-key !’ non-key dependency) '
     hospitals: hospital_id !’ name, address (not via node_id) '
     hospital_specializations: spec_id !’ {hospital_id, emergency_type} only '
```

Key SQL Queries Used

```
-- Filter hospitals by emergency type
SELECT h.* FROM hospitals h
JOIN hospital_specializations hs ON h.hospital_id = hs.hospital_id
WHERE hs.emergency_type = 'CARDIAC';

-- Check bed availability
SELECT icu_beds_available, general_beds_available
FROM hospital_resources WHERE hospital_id = 'H3';

-- Record new allocation
INSERT INTO allocations (case_id, hospital_id, route_cost_km, path_nodes)
VALUES (42, 'H3', 3.48, '[9, 5]');
```

What to Say in Viva (BCS403)

localStorage as DBMS Demo

"In our implementation, we use localStorage as the persistence layer — this is equivalent to a client-side database. The emergency_history key acts as our allocations table, hospital_beds acts as our hospital_resources table. The schema I designed is fully normalized to 3NF — I can explain any table's primary key, foreign keys, and why there are no partial or transitive dependencies."

Slide 6 — BCS405B: Graph Theory

BCS405B

City as a weighted undirected graph — traversal and shortest path proofs

Graph Definition — G = (V, E)

```
V = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 }    |V| = 13
E = 23 undirected weighted edges (e1 through e24, no e12)
w(u,v) = road distance in km  ×  traffic_multiplier (1.0 – 2.0)

Properties:
  - Undirected: roads are two-way    !’ bidirectional in adjacency list
  - Weighted:   edge weights "e 0    !’ Dijkstra valid (no negative cycles)
  - Connected:  all 13 nodes reachable from any source
  - Simple:     no self-loops, no multi-edges between same pair
```

Graph Representation — Adjacency List

```
Node 1 (JSSATE) !’ [ (3, 6.2km, ×1.0), (5, 1.2km, ×1.0), (6, 5.8km, ×1.5), (8, 7.2km, ×1.2), (9,
3.8km, ×1.0) ]
Node 5 (BGS)    !’ [ (1, 1.2km, ×1.0), (9, 2.9km, ×1.2) ]
Node 9 (Kengeri)!’ [ (1, 3.8km, ×1.0), (5, 2.9km, ×1.2), (8, 4.8km, ×1.0) ]
...
Space complexity: O(V + E) = O(13 + 46) = O(59) directed adjacency entries
```

BFS — Breadth-First Search

- Queue-based level-order traversal: explores all nodes at distance d before nodes at distance d+1.
- Used for: connectivity check (is hospital H reachable from node X?).
- Answer: YES/NO — does not give shortest weighted distance.
- Complexity: O(V + E) = O(36) for this graph.

DFS — Depth-First Search

- Stack-based (or recursive) deep exploration: follows one path to the end before backtracking.
- Used for: listing ALL reachable hospitals from a source — complete reachability analysis.
- Demonstrates backtracking, recursion, and graph connectivity concepts.
- Complexity: O(V + E) = O(36) for this graph.

What to Say in Viva (BCS405B)

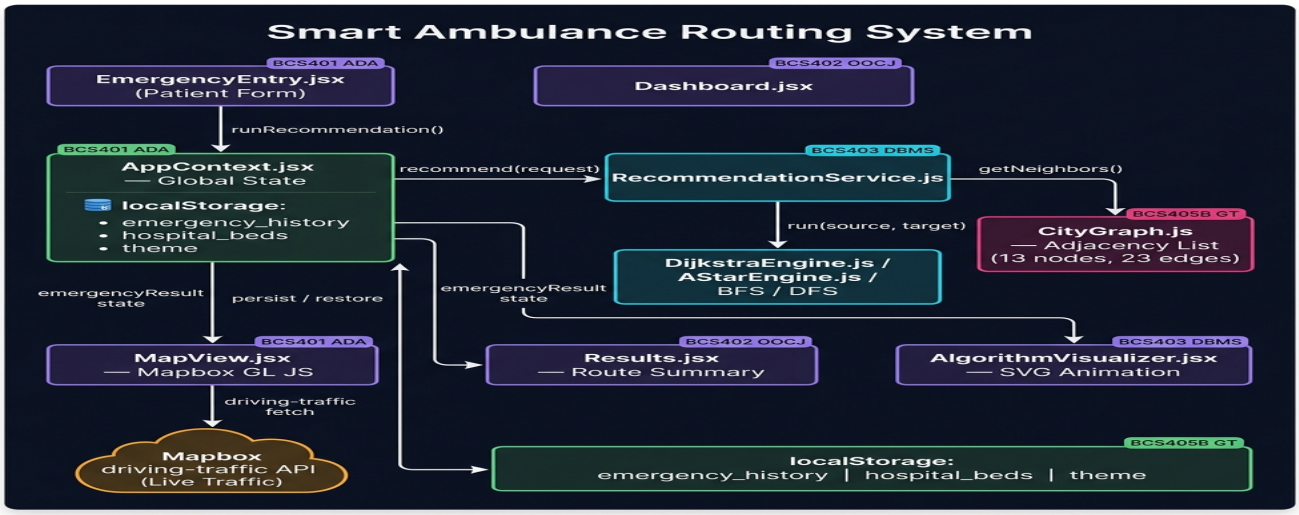
Examiner Question: Why adjacency list and not adjacency matrix?

"Our city graph has 13 nodes but only 23 edges — it is sparse. An adjacency matrix would need 13×13 = 169 cells, but only 46 (23×2) would be non-zero. An adjacency list uses exactly V + 2E = 59 entries — much more memory-efficient for sparse graphs. Also, Dijkstra's getNeighbors() operation is O(degree) with adjacency list vs O(V) with matrix."

Slide 7 — System Architecture

SLIDE 7

Component layers, data flow, and the architecture diagram explained



4-Layer Architecture

1. **UI Layer (React JSX pages):** Dashboard, EmergencyEntry, Results, MapView, AlgorithmVisualizer, Hospitals, History, ERDiagram, About — each is a self-contained React component
2. **Context / State Layer:** AppContext.jsx — single source of truth. Holds hospitals, history, emergencyResult, theme. Persists beds and history to localStorage.
3. **Engine Layer:** CityGraph (adjacency list), PriorityQueue (binary min-heap), DijkstraEngine, AStarEngine, BFSEngine, DFSEngine, RecommendationService
4. **Data Layer:** cityData.js — static database of 13 nodes, 23 edges, 6 hospitals, ROUTE_COORDS, NODE_POSITIONS. Read-only; never mutated at runtime.

External Integration: Mapbox API

- MapView.jsx calls the Mapbox Directions API with profile: driving-traffic (not static 'driving').
- Returns: GeoJSON route geometry + per-segment congestion annotation ['low','moderate','heavy','severe'].
- Each road segment is colored independently: green=low, amber=moderate, red=heavy, dark red=severe.
- Ambulance animation uses Turf.js: along(route, distance) gives point at each animation frame.
- Bearing offset: setRotation(bearing " 90) because Ø=Þ‘ emoji faces east by default (90°), Mapbox 0° = north.

Slide 8 — How All 4 Subjects Work Together

The interdependency chain that makes the system work

SLIDE 8

The Dependency Chain

Each subject is not standalone — they form a chain where the output of one subject feeds the next. This is the key insight that makes this project exam-ready:

1. **BCS403 DBMS defines the data structures:** The ER diagram tells us WHAT to store — patients, hospitals, roads, specializations, resources. Without DB schema design, we don't know what fields to query.
2. **BCS405B Graph Theory models the data:** The hospital table gives us nodes; the road_edges table gives us edges and weights. Graph Theory takes the DBMS schema and turns it into G=(V,E).
3. **BCS401 ADA runs on the graph:** Dijkstra's SSSP runs on G=(V,E) from DBMS+GraphTheory. The algorithm uses edge weights from road_edges and the filter conditions from hospital_specializations.
4. **BCS402 OOCJ/Java builds the whole system:** The application code (CityGraph, DijkstraEngine, RecommendationService, AppContext, React pages) ties everything together. OOP structure makes each piece reusable and testable.

Subject Roles in the Recommendation Pipeline

Subject !' Role	Code Location
BCS403: defined hospital_specializations table	cityData.js HOSPITALS[].specializations[]
BCS403: hospital_resources has bed counts	AppContext.jsx hospitals[].icuBedsAvailable
BCS405B: graph G = road_edges as adjacency list	CityGraph.js addEdge() + getNeighbors()
BCS401: Dijkstra SSSP finds min-cost path	DijkstraEngine.js run(source, target)
BCS402: RecommendationService orchestrates all	RecommendationService.js recommend(request)

What to Say in Viva (Slide 8)

Connecting All 4 Subjects

"The beauty of this project is that all 4 subjects are genuinely interdependent — not just independently demonstrated. DBMS gives us the schema. Graph Theory converts the schema into a graph. Algorithm Design finds the optimal path on that graph. And Advanced Java / OOP builds the application that ties it all together. Each subject is incomplete without the others."

Slide 9 — 3 Demo Cases

SLIDE 9

Walkthrough of three test scenarios to explain during viva

CASE 01 — Nearest Hospital IS Best (Simple Case)

CASE 01

Scenario: Patient at Node 1 (JSSATE) | Emergency: GENERAL | No ICU

- Specialization filter: All 6 hospitals pass (all have GENERAL)
- Bed filter: All 6 pass (generalBedsAvailable > 0)
- Dijkstra from Node 1: BGS (Node 5) = 1.2km, Sagar (Node 4) = 7.7km, RRMCH (Node 8) = 7.2km...
- BGS wins — minimum cost path [1 !' 5] at 1.2 km

Result: BGS Gleneagles Hospital selected — 1.2 km, ETA ~1.4 minutes.

& Demonstrates system works correctly in the simple case. Dijkstra confirms the nearby obvious choice.

CASE 02 — Nearest Hospital Lacks Specialization

CASE 02

Scenario: Patient at Node 1 (JSSATE) | Emergency: TRAUMA | Needs ICU

- Specialization filter: TRAUMA hospitals = RRMCH (H1), Sagar (H2), Fortis (H6) — BGS is filtered OUT
- Bed filter: All 3 pass (icuBedsAvailable > 0)
- Dijkstra: RRMCH (Node 8) = 7.2 km via [1!'8], Sagar (Node 4) = 7.7 km via [1!'3!'4], Fortis = 17+ km
- RRMCH wins — minimum cost path [1 !' 8] at 7.2 km

Result: RRMCH Mysore Road selected (7.2 km) even though BGS is only 1.2 km — because BGS doesn't handle TRAUMA.

& KEY VIVA POINT: Shows why 'nearest' "` 'best'. The specialization constraint is the critical differentiator.

CASE 03 — Multiple Valid Hospitals — Dijkstra Decides

CASE 03

Scenario: Patient at Node 10 (BSK Temple) | Emergency: CARDIAC | Needs ICU

- Specialization filter: CARDIAC = BGS (H3), Apollo (H5), Fortis (H6) — 3 candidates
- Bed filter: All 3 have ICU beds (2, 5, 4 respectively) — all pass
- Dijkstra from Node 10: BGS via [10!'17!'7!'...!'5] "H 9+ km, Apollo via [10!'16!'2!'11] "H 5.5 km, Fortis via [10!'16!'2!'13] "H 5
- Apollo wins — minimum cost path at ~5.5 km

Result: Apollo Hospital Bannerghatta Road selected — demonstrating Dijkstra choosing the globally optimal hospital.

& BEST CASE FOR VIVA: Three hospitals all qualify. Dijkstra is the ONLY way to decide which is truly nearest via roads (not straight line).

Slide 10 — Viva Closing

SLIDE 10

The one-liner to remember and subject role summary

The One Line to Remember

Master Quote for Viva

"This project uses DBMS to store hospitals and roads, Advanced Java to build the application, Graph Theory to model and solve routing, and Algorithm Design to compare and justify the recommendation strategy."

Each Subject's Role (4 Chips)

1. **BCS401 — Defined the Problem:** Justified why Dijkstra over brute-force. Compared BFS/DFS/Dijkstra/A* complexities. Proved optimality.
2. **BCS402 — Built the App:** All OOP classes (CityGraph, DijkstraEngine, RecommendationService, PriorityQueue). React component architecture. localStorage as JDBC.
3. **BCS403 — Designed the DB:** 8-table ER diagram in 3NF. hospital_specializations, hospital_resources, emergency_cases, allocations tables.
4. **BCS405B — Solved the Routing:** Weighted undirected graph G=(V=13, E=23). Adjacency list. BFS/DFS traversal. Dijkstra + A* shortest path.

Likely Viva Questions & Short Answers

Q: What is the time complexity of Dijkstra?

A: O((V+E) log V) using a binary min-heap priority queue. For our graph: O((13+23) × log 13) "H O(133).

Q: Why not use A* always?

A: A* needs a consistent heuristic. Our Haversine heuristic is admissible, so A* is correct. But for 13 nodes the speed gain is minimal — Dijkstra is simpler.

Q: Where is data stored?

A: In localStorage on the browser — no backend server. emergency_history and hospital_beds keys persist across sessions.

Q: What is 3NF?

A: No partial or transitive dependencies. Every non-key attribute depends directly on the primary key — proven for all 8 tables.

Q: How does live traffic work?

A: Mapbox driving-traffic API returns route geometry + congestion annotations per segment. We color each segment green/amber/red accordingly.

End-to-End Architecture Flow — Part 1

ARCH

From app startup to emergency result: complete function call trace

Phase 1: Application Bootstrap

```
Browser loads index.html !' <script type="module" src="src/main.jsx">

main.jsx:
  ReactDOM.createRoot(root).render(
    <AppProvider>        !• wraps EVERYTHING in global context
      <App />            !• Router + Navbar + all page Routes
    </AppProvider>
  )

AppProvider mounts (AppContext.jsx):
  graph = useMemo(() => {
    g = new CityGraph()
    LOCATIONS.forEach(loc => g.addNode(loc.id, loc))  // 13 nodes added
    EDGES.forEach(e => g.addEdge(e.from, e.to, e.weight, {
      roadName, trafficMultiplier, id                  // 23 bidirectional edges
    }))
    return g  // adjacency Map: 13 keys, each with neighbor array
  }, [])

  hospitals = useState(() => {                         // init from localStorage
    saved = localStorage.getItem('hospital_beds')
    if (saved): merge saved bed counts into HOSPITALS base data
    else: HOSPITALS.map(h => ({...h}))                 // 6 hospitals
  })

  history = useState(() => {                           // init audit log
    saved = localStorage.getItem('emergency_history')
    return saved ? JSON.parse(saved) : []
  })

  theme = useState(() => localStorage.getItem('theme') || 'dark')
  useEffect([theme] !' document.documentElement.setAttribute('data-theme', theme))
```

Phase 2: Emergency Form Submission

```
User navigates to /emergency !' EmergencyEntry.jsx mounts
User selects: sourceLocationId (parseInt on change !' always number type)
              emergencyType = 'CARDIAC'
              needsIcu = true
              patientName = "Shreya"

handleSubmit(event):
  event.preventDefault()
  if (!patientName): alert(); return
  runRecommendation({
    sourceNodeId: parseInt(9),    !• always number, not string
    emergencyType: 'CARDIAC',
    needsIcu: true,
    patientName: 'Shreya',
    useTraffic: true
  })
```

End-to-End Architecture Flow — Part 2

ARCH

Dijkstra execution, result storage, and map rendering

Phase 3: Recommendation Engine (AppContext !’ RecommendationService !’ Dijkstra)

```
AppContext.runRecommendation(request):
  setEmergencyRequest(request)
  service = new RecommendationService(graph, hospitals)
  result = service.recommend(request)

RecommendationService.recommend({ sourceNodeId:9, emergencyType:'CARDIAC', needsIcu:true }):

  STAGE 1 — Specialization Filter:
    candidates = hospitals.filter(h => h.specializations.includes('CARDIAC'))
    !’ [BGS(H3), Apollo(H5), Fortis(H6)]  — 3 pass, 3 filtered out

  STAGE 2 — Bed Availability Filter:
    candidates = candidates.filter(h => h.icuBedsAvailable > 0)
    !’ [BGS(icuAvail=2), Apollo(icuAvail=5), Fortis(icuAvail=4)]  — all 3 pass

  STAGE 3 — Dijkstra per candidate (sourceNode = 9):

    For BGS (nodeId=5):
      pq = PriorityQueue()
      pq.insert(9, 0)                          dist = {9:0, all others:" }
      iter1: u=9 (dist=0), not visited
        neighbor 8: w=4.8×1.0=4.8   !’ dist[8]=4.8, prev[8]=9
        neighbor 1: w=3.8×1.0=3.8   !’ dist[1]=3.8, prev[1]=9
        neighbor 5: w=2.9×1.2=3.48  !’ dist[5]=3.48, prev[5]=9  !• traffic!
      iter2: extractMin !’ u=5 (dist=3.48)  !• TARGET FOUND
      path = [9, 5],  cost = 3.48 km

    For Apollo (nodeId=11):   Dijkstra finds path 9!’1!’3!’...!’11 "H longer
    For Fortis  (nodeId=13):  Dijkstra finds path "H longer still

  Sort by cost !’ [BGS(3.48km), Apollo, Fortis]

  return { primary: BGS, backup: Apollo, filterSteps, explanation }

setEmergencyResult(result)
addToHistory({ ...entry, needsIcu: request.needsIcu })
  !’ setHistory(prev => [newEntry, ...prev])
  !’ useEffect([history]) !’ localStorage.setItem('emergency_history', JSON.stringify(history))
```

End-to-End Architecture Flow — Part 3

ARCH

Results display, live traffic map fetch, and ambulance animation

Phase 4: Results Display (Results.jsx)

```
Results.jsx reads: { primary, backup, filterSteps, explanation } = emergencyResult

Renders:
  Patient bar: "Shreya (CARDIAC • ICU) — From Kengeri Bus Terminal"
  Primary card:
    BGS Gleneagles Global Hospital
    ETA = Math.round((3.48 / 50) * 60) = 4 min   !• 50 km/h emergency speed
    Distance: 3.48 km
    Path: Node 9 !’ Node 5
    Explanation: "Selected BGS because..."
  Backup card: Apollo Hospital (longer path)
  Filter steps timeline: Specialization(3) !’ Beds(3) !’ Shortest Path(BGS wins)
  Buttons: "View on Map" !’ /map    "Visualize Algorithm" !’ /visualizer
```

Phase 5: Live Traffic Map Route (MapView.jsx)

```
MapView.jsx mounts !’ useEffect fires:
  map = new mapboxgl.Map({
    style: 'mapbox://styles/mapbox/streets-v12',
    center: [77.5250, 12.9080],  zoom: 12.5
  })

  map.on('load'):
    Draw 23 graph edges as dark-grey GeoJSON LineStrings
    Add distance badge markers at midpoint of each edge

    if emergencyResult.primary: fetchAndDrawRoute(map, [9, 5])

fetchAndDrawRoute(map, path=[9,5]):
  pathCoordsString = "77.4810,12.9115;77.4984,12.8985"

  fetch('https://api.mapbox.com/directions/v5/mapbox/driving-traffic/
         77.4810,12.9115;77.4984,12.8985
         ?geometries=geojson&overview=full&annotations=congestion,duration
         &access_token=pk.eyJ1...')

  Response: {
    routes: [{
      distance: 3250 meters,
      duration: 420 seconds,
      geometry: { type:'LineString', coordinates: [[77.481,12.911], ...] },
      legs: [{ annotation: { congestion: ['low','low','moderate',...] } }]
    }]
  }

  setLiveRouteData({ distance:'3.25km', duration:7min, dominantCongestion:'low' })

  // Per-segment colored route:
  CONGESTION_CONFIG:
    low      !’ #10b981 (green)
    moderate !’ #f59e0b (amber)
    heavy    !’ #ef4444 (red)
    severe   !’ #dc2626 (dark red)

  addLayer('highlight-route-bg',         glow cyan bg,  opacity 0.18, blur 8)
  addLayer('highlight-route-congestion', per-segment colors via 'line-color': ['get','color'])
```

Phase 6: Ambulance Animation (Turf.js + requestAnimationFrame)

```
runAnimation():
  pathCoords = mapRouteGeometryRef.current.coordinates   // from Mapbox API
  route = turf.feature(LineString(pathCoords))
  lineDistance = turf.length(route, { units: 'kilometers' })  // 3.25 km

  Ø=Þ‘ marker = new mapboxgl.Marker({ element: div('Ø=Þ‘'), rotationAlignment:'map' })
  marker.setLngLat(pathCoords[0]).addTo(map)
  map.fitBounds(allCoords, { padding:80, duration:1000 })

  startTime = null
  requestAnimationFrame(animate)

  animate(timestamp):
    startTime = startTime || timestamp
    progress = min((timestamp - startTime) / 8000, 1)   // 8 second animation
    distance = progress × lineDistance

    point = turf.along(route, distance, { units:'km' })
    marker.setLngLat(point.geometry.coordinates)

    // &  BEARING FIX: Ø=Þ‘ emoji faces EAST (90°) by default
    //   Mapbox setRotation(0) = NORTH
    //   So subtract 90° to align emoji front with travel direction
    prevPoint = turf.along(route, max(0, distance - 0.025))
    bearing = turf.bearing(prevPoint, point)
    marker.setRotation(bearing - 90)

    if progress < 1: requestAnimationFrame(animate)
    else: setIsAnimating(false)
```